

COMS 3003A

Test 2

3 May, 2024

This is a closed book test.
Time allowance: 60 minutes

1. (a) (10 points) **Prove, using diagonalization, that the self-rejection problem for Turing machines is algorithmically unsolvable, i.e., that the language $SR = \{\langle M \rangle \mid M(\langle M \rangle) = 0\}$ is undecidable. You may not assume unsolvability of any other problem.**

Suppose, for the sake of contradiction, that this problem is solvable. Then, there exists a Turing machine, say M_{SR} , that solves it, i.e., such that

$$M_{SR}(\langle M \rangle) = 1 \iff M(\langle M \rangle) = 0.$$

Then, in particular,

$$M_{SR}(\langle M_{SR} \rangle) = 1 \iff M_{SR}(\langle M_{SR} \rangle) = 0,$$

which is impossible.

- (b) (10 points) **Prove, using unsolvability of the problem from (a), that the problem of determining whether a Turing machine rejects an input word is unsolvable, i.e., that the language $R = \{\langle M, w \rangle \mid M(w) = 0\}$ is undecidable.**

Suppose, for the sake of contradiction, that R is decidable. Then, there exists a Turing machine, say M_R , that decides it. Using M_R , we can define a Turing machine M_{SR} that decides the language SR :

- (1) M_{SR} , on input $\langle M \rangle$, calls M_R with arguments $\langle M \rangle$ and $\langle M \rangle$.
- (2) If $M_R(\langle M \rangle, \langle M \rangle) = 1$, then M_{SR} accepts; if $M_R(\langle M \rangle, \langle M \rangle) = 0$, then M_{SR} rejects.

Let's show that M_{SR} , indeed, decides SR :

- If $M_{SR}(\langle M \rangle) = 1$, then $M_R(\langle M \rangle, \langle M \rangle) = 1$, which means, by specification of M_R , that $M(\langle M \rangle) = 0$.
- If, on the other hand, $M_{SR}(\langle M \rangle) = 0$, then $M_R(\langle M \rangle, \langle M \rangle) = 0$, which means, by specification of M_R , that $M(\langle M \rangle) \neq 0$.

2. (10 points) Prove that the problem of determining whether a Turing machine ever writes a 1 is algorithmically unsolvable. You may assume that the following problems are unsolvable:

- self-acceptance for Turing machines;
- acceptance for Turing machines;
- halting for Turing machines.

Suppose, for the sake of contradiction, that this problem is solvable. Then, there exists a Turing machine, say M_1 , that solves it. Then, we can define a Turing machine M_A that decides acceptance:

- (1) M_A , on input $\langle M, w \rangle$, writes down (the program of) a Turing machine $M_{M,w}$ that accepts no input and works as follows:
 - (a) $M_{M,w}$ simulates M on w in such a way that it never writes a 1 (surely, $M_{M,w}$ can always avoid writing a 1: if it needs to write a 1, it can write a special tape symbol, say $\#$, instead);
 - (b) If $M(w) = 1$, then $M_{M,w}$ writes a 1 and accepts; if $M(w) = 0$, then $M_{M,w}$ immediately rejects.
- (2) M_A calls M_1 with the argument $\langle M_{M,w} \rangle$.
- (3) If $M_1(\langle M_{M,w} \rangle) = 1$, then M_A accepts; $M_1(\langle M_{M,w} \rangle) = 0$, then M_A rejects.

Let's show that M_A , indeed, decides acceptance:

- If $M_A(\langle M \rangle, w) = 1$, then $M_1(\langle M_{M,w} \rangle) = 1$, which means, by specification of M_1 , that $M_{M,w}$ writes a 1 (since $M_{M,w}$ accepts no inputs, it always performs the same computation when it runs). This only happens if $M(w) = 1$.
- If $M_A(\langle M \rangle, w) = 0$, then $M_1(\langle M_{M,w} \rangle) = 0$, which means, by specification of M_1 , that $M_{M,w}$ never writes a 1. This happens if $M(w) = \infty$ or if $M(w) = 0$, i.e., if $M(w) \neq 1$.

Since acceptance is unsolvable, however, M_A does not exist, which gives us a contradiction.

3. (a) (5 points) Prove that the set of Turing machines is countable.

We know that every Turing machine is encodable as a string in a finite alphabet and that the set of strings in a finite alphabet is countable. Hence, there exists an injection (an encoding function) from the set of Turing machines to a countable set (of finite strings). Hence, the set of Turing machines is countable.

- (b) (10 points) Prove that the set of decision problems is uncountable.

A decision problem is a function from the set Σ^* of finite strings in a finite alphabet (which we may assume to be binary) to the set $\{0, 1\}$. Hence, we need to show that the set F of all such functions is uncountable.

Suppose otherwise, i.e., suppose that F is countable. Then, there exists a bijection between F and Σ^* . Denote a function that is bijectively mapped to $w \in \Sigma^*$ by f_w . Define a function f_D as follows:

$$f_D(w) := \begin{cases} 1 & \text{if } f_w(w) = 0; \\ 0 & \text{if } f_w(w) = 1. \end{cases}$$

Then, f_D is a function from Σ^* to $\{0, 1\}$; hence, $f_D \in F$. On the other hand, it is not hard to see that $f_D \notin F$: otherwise, f_D is bijectively mapped to some $d \in \Sigma^*$, and hence, by definition of f_D ,

$$f_D(d) = 1 \iff f_D(d) = 0,$$

which is impossible.

- (c) **(5 points)** What conclusion can we draw from (a) and (b) about undecidable problems? Explain your answer.

We can conclude that there exist undecidable problems (even though we do not know which problems exactly are undecidable). Indeed, if all the problems were decidable, then there would exist a surjection from the set of Turing machines that are deciders onto the set of all decision problems (mapping a decider to the problem it decides). But then the set of decision problems would be not larger than the set of deciders, and hence not larger than the set of all Turing machines, which contradicts (a) and (b).

4. The first-order language of arithmetic contains the constant 0, the unary function symbol S (successor), and the binary predicate symbols $\cdot$ and $+$. Translate the following statements into this language (you are free to define any other symbols in terms of the ones listed above):

- (a) **(5 points)** Every natural number is divisible by 1.

$$\forall x \exists y (y \cdot S(0) = x).$$

- (b) **(5 points)** There is no biggest natural number.

First, we define the relation $<$:

$$x < y := \exists z (\neg(z = 0) \wedge x + z = y).$$

Next, we say what we want to say:

$$\forall x \exists y x < y.$$

Of course, we can combine the two:

$$\forall x \exists y \exists z (\neg(z = 0) \wedge x + z = y).$$